

Centralized Threat Detection Through Wazuh–Snort SIEM Integration

A SOC Lab Project Report

Prepared by: Amjad Khaleel Farhan

Date: 29 June 2026

Detect • Forward • Correlate • Visualize

Table of Contents

1. Abstract
2. Project Objectives
3. Lab Architecture
 - 3.1 Virtual Machine Roles
 - 3.2 Resource Allocation
 - 3.3 Network Topology
4. Data Flow, Alert Flow, and Detection Architecture
 - 4.1 End-to-End Pipeline
 - 4.2 Why This Matters Operationally
5. Implementation Methodology
 - 5.1 Phase 1 — Wazuh Agent Deployment
 - 5.1.1 Repository Trust Setup
 - 5.1.2 Agent Installation and Manager Linkage
 - 5.1.3 Agent Enrollment (Key Exchange)
 - 5.2 Phase 2 — Wazuh ↔ Snort Log Integration
 - 5.2.1 Identifying Snort's Output Format
 - 5.2.2 Configuring the Wazuh Agent to Monitor the Log
 - 5.3 Phase 3 — Enabling Active Detection and Generating Attack Traffic
 - 5.3.1 Enabling the Portscan Preprocessor
 - 5.3.2 Generating Attack Traffic
6. Troubleshooting Log and Technical Findings
 - 6.1 GPG Keyring Syntax Error
 - 6.2 Self-Directed Scan Traffic Invisible to Snort
 - 6.3 Host RAM Constraints and the Kali Linux Attacker VM
 - 6.4 Malformed ossec.conf XML Structure
 - 6.5 Wazuh-Indexer (OpenSearch) Out-of-Memory Crash
 - 6.6 Agent Re-enrollment Producing a Duplicate Identity
7. Results and Verification
 - 7.1 The Test Attack
 - 7.2 Stage 1 — Raw Snort Detection
 - 7.3 Stage 2 — Wazuh Dashboard Visualization
8. SOC-Style Alert Analysis
9. Security Best Practices Demonstrated
10. Real-World SOC Relevance
11. Conclusion
 - 11.1 Future Improvements
12. Appendix: Figure Index

1. Abstract

This project demonstrates the practical integration of Snort, an open-source network intrusion detection system (IDS), with Wazuh, an open-source Security Information and Event Management (SIEM) platform, within a self-contained virtual lab environment. The objective was to build a working pipeline in which network-based attacks are detected at the packet level by Snort, forwarded as structured log data through a Wazuh Agent, centrally correlated by a Wazuh Manager, and finally visualized for analyst review on the Wazuh Dashboard.

The lab consisted of three virtual machines orchestrated in Oracle VirtualBox: a Wazuh Manager (SIEM server), an Ubuntu victim host running Snort and a Wazuh Agent, and an originally-planned Kali Linux attacker machine. Over the course of the build, the project encountered and resolved a series of real-world operational issues — including package signing failures, VirtualBox network isolation, agent re-enrollment conflicts, malformed configuration files, host memory exhaustion, and a non-obvious Linux networking behavior that rendered self-directed scan traffic invisible to the IDS. Each issue was diagnosed methodically using log analysis and first-principles networking knowledge rather than trial-and-error guessing.

The final outcome is a fully verified, end-to-end detection pipeline: a TCP SYN scan launched against the victim host was detected by Snort, classified as an SNMP information-leak attempt, written to Snort's alert log, ingested by the Wazuh Agent, transmitted to the Wazuh Manager, and successfully rendered as a structured event in the Wazuh Dashboard — complete with agent attribution, severity level, and rule metadata. This report documents the architecture, implementation methodology, troubleshooting process, and final validation of that pipeline, and reflects on its relevance to real-world Security Operations Center (SOC) practice.

2. Project Objectives

The project was designed around the following core objectives:

- Deploy a centralized SIEM platform (Wazuh) capable of receiving and correlating telemetry from a remote endpoint.
- Deploy a network intrusion detection system (Snort) on a monitored endpoint, capable of inspecting live network traffic.
- Integrate Snort's alert output into the Wazuh Agent's log collection pipeline, so that IDS detections are forwarded to the SIEM in real time.
- Generate genuine, realistic attack traffic against the monitored endpoint and confirm that the detection-to-visualization pipeline functions correctly end-to-end.
- Practice SOC-relevant operational skills: log analysis, root-cause troubleshooting, resource-constrained lab management, and security configuration hardening.
- Document the build process, architecture, and findings in a format suitable for academic submission, technical presentation, and viva examination.

3. Lab Architecture

The lab was built entirely within Oracle VirtualBox, using three virtual machines connected via a shared NAT Network (10.0.2.0/24). This configuration allows full mutual connectivity between guest virtual machines while isolating them from the host machine's network by default — a detail that becomes operationally significant later in this report (see Section 6.3).

3.1 Virtual Machine Roles

Machine	Role	IP Address	Key Software
Wazuh-Manager	SIEM Server	10.0.2.12	Wazuh Manager, Wazuh Dashboard, OpenSearch (Indexer)
Ubuntu-Victim	Monitored Endpoint	10.0.2.14	Snort IDS, Wazuh Agent
Kali Linux	Originally-planned Attacker	N/A (unused)	Nmap, standard pentest toolkit

3.2 Resource Allocation

Virtual machine resources were tuned over the course of the project to operate within an 8 GB host RAM budget. Final allocations:

Machine	Base Memory	Video Memory	Processors
Wazuh-Manager	3072 MB	16 MB	2
Ubuntu-Victim	1536 MB	16 MB	1
Kali Linux	1536 MB (reduced from 2048 MB)	16 MB	1 (reduced from 2)

Note: due to host RAM constraints, the Kali Linux VM was ultimately not run concurrently with the other two machines for the final detection test. This constraint, and the alternative approach adopted, is discussed in Section 6.2.

3.3 Network Topology

All three virtual machines reside on the same VirtualBox NAT Network and can reach one another directly. The Windows host accesses the Wazuh Dashboard through a VirtualBox port-forwarding rule, since NAT Network traffic is isolated from the host by default.

Connection	Protocol / Port	Purpose
Victim → Manager	TCP 1514	Wazuh Agent event forwarding
Victim ↔ Manager	TCP 1515	Wazuh Agent enrollment / key exchange
Host → Manager	TCP 8443 (host) → 443 (guest)	Wazuh Dashboard access via port forwarding
Manager → Victim	TCP/ICMP (varies)	Test attack traffic (Nmap scan)

[illegible]

Fig. 1: Wazuh Manager terminal showing network interface configuration (ip a) confirming IP 10.0.2.12

```
ubuntu22.04.5 LTS ubuntu-victim tty1

ubuntu-victim login: [ 23.234233] cloud-init[1264]: Cloud-init v. 25.1.4-0ubuntu0~22.04.1 running
'modules:config' at Sun, 28 Jun 2026 06:30:08 +0000. Up 22.98 seconds.
[ 24.841791] cloud-init[1341]: Cloud-init v. 25.1.4-0ubuntu0~22.04.1 running 'modules:final' at Su
n, 28 Jun 2026 06:30:10 +0000. Up 24.71 seconds.
[ 24.892446] cloud-init[1341]: Cloud-init v. 25.1.4-0ubuntu0~22.04.1 finished at Sun, 28 Jun 2026
06:30:10 +0000. DataSource DataSourceNone. Up 24.88 seconds

ubuntu-victim login: socanalyst
Password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-181-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun Jun 28 06:30:35 AM UTC 2026

System load: 0.84          Processes: 113
Usage of /: 49.4% of 11.21GB Users logged in: 0
Memory usage: 22%         IPv4 address for enp0s3: 10.0.2.14
Swap usage: 0%

Expanded Security Maintenance for Applications is not enabled.

77 updates can be applied immediately.
11 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Sat Jun 27 03:05:10 UTC 2026 on tty1
socanalyst@ubuntu-victim:~$
```

Fig. 2: Ubuntu Victim terminal showing network interface configuration confirming IP 10.0.2.14

4. Data Flow, Alert Flow, and Detection Architecture

Understanding how data moves through this pipeline is essential to both the implementation and the troubleshooting described later in this report. The complete flow, from attack to dashboard visualization, is as follows:

4.1 End-to-End Pipeline

1. Traffic generation: A network scan (Nmap SYN scan) is sent across the network toward the Ubuntu-Victim machine.
2. Packet-level detection: Snort, running in NIDS (Network Intrusion Detection System) mode on the victim's real network interface (enp0s3), inspects every packet crossing that interface against its loaded rule set and preprocessors.
3. Local logging: When a rule or preprocessor matches, Snort writes a structured, single-line alert to its plain-text log file, `/var/log/snort/snort.alert.fast`, using the `alert_fast` output format.
4. Log collection: The Wazuh Agent's logcollector module, configured via a `<localfile>` directive in `ossec.conf`, continuously tails this file and parses each new line using Wazuh's built-in `snort-fast` decoder.
5. Forwarding: Parsed events are transmitted securely over TCP port 1514 to the Wazuh Manager, authenticated using the agent's pre-shared key (established during enrollment).
6. Correlation: The Wazuh Manager's analysis engine applies its rule base to the incoming event, assigning a rule ID, severity level, and classification (e.g., "First time this IDS alert is generated").
7. Indexing: The correlated alert is stored in the OpenSearch backend (Wazuh Indexer), making it searchable and persistent.
8. Visualization: The Wazuh Dashboard queries the Indexer and renders the alert in the Threat Hunting / Events view, where it can be filtered, inspected, and reported on by an analyst.

4.2 Why This Matters Operationally

This is precisely the model used in real-world Security Operations Centers: dozens or hundreds of endpoints each run a lightweight agent and a local detection tool, but no analyst manually inspects individual host logs. Instead, every signal is centralized into a single SIEM, where correlation rules, dashboards, and search tooling allow a small team to monitor an entire estate from one pane of glass. This project reproduces that exact architecture at small scale, including the same trust and authentication model (agent enrollment via shared keys) used in production Wazuh deployments.

5. Implementation Methodology

The build was carried out in five sequential phases, each validated before proceeding to the next.

5.1 Phase 1 — Wazuh Agent Deployment

The Wazuh Agent was installed on the Ubuntu-Victim machine and connected to the Wazuh Manager, following these steps:

5.1.1 Repository Trust Setup

Before installing any Wazuh package, the system was configured to trust Wazuh's official package repository. This required importing Wazuh's GPG public signing key into a dedicated keyring:

```
sudo mkdir -p /usr/share/keyrings
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH -o /tmp/wazuh-key.gpg
sudo gpg --no-default-keyring \
  --keyring gnupg-ring:/usr/share/keyrings/wazuh.gpg \
  --import /tmp/wazuh-key.gpg
sudo chmod 644 /usr/share/keyrings/wazuh.gpg
```

The repository was then registered and verified:

```
echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main" | sudo
tee -a /etc/apt/sources.list.d/wazuh.list
sudo apt update
```

```
TpKSP/XYCTBhFa390rYgF021bLDVsiz7Txd94nHrdWX6EQfurbsvd1LLWk7iN81
Fb4B600fRi3yoR96a/kIPNa0x26+n79LTDuWZ/DtQ5JSHztdd9F1sr3h815zYmtv
luJ992orpuYeJbBVuMo+gP01oaXM37u056UFVQk3po9GaS+GtLn1goE5vo1gNYy0
rkeIua4u2Vs1fREkHCKoLJip6P7S3kTyfrpISLhouE27kV11bMbFgvHXyJm+/AIX
HIBg++H+e+HNT5gZzTKUJSuBJx44+4JYSOR67EJ0dtP0pgiuJXhedzShE06rbu/04
wMirx45ZXDYa2FGb1HCQ/VaS0ttFtztK91xw1WvjTR8vGvp5tIfC1+1G1xPRQpbN
Y/oo8kv4A7vB3J1JscJC1jvRgaX0GTBz1aF6Gg0FdcWE15F1zvsHCSc/Fv5WrUPY
5mG0m69YUTEV06cZS1aiu9Qh3QAT/7NbUuGXIaAXKnu+kKjLSz+nTT10yvbG7BVF
a6SDmw48qIcebkC/rCt04g8107KoA2XC/K/6PAxDrLkVyu8WPSAendmezNfHU+V
32bvWoQoQgu8ysoaEYc/J9fN4H3mEBN3QUJYCugmHP0pu7VtpWuWUgcGeUVuAR
AQAB1QI1BBgBCAAPAhsMBQJ2HN0aBQkUSg0HAAoJEJaz718pERFFhpkQAJO9mjJp
n9f18JGSMzP41fVucPuLB25XJL/hy2boI1Fvgfm0ETzNxLPb1HdkJVJ2S51MrhL
EJ1jv+GQDtf68/0J0+HXuQIBmUJ53YwbuuQLWH7CI2Ax1SAKAn2k0ApMMKSjnAv
JwS3eNGukOKRfEKtQz2Vw11H7M7ppyp29KeoyAoSIWb61gm7rXbfT+TVetHfrU
EM5vz3AS3pJk6Yfn10IZf1exXmsBD+SpJBNzMBsZnCcW02y4q2NLJfFerBoizvV
34Un2ydlbkSN0T/HKp8sgJwqDJBS72tH62IM8NNoy29aPDKaa8X1hkW1BdRizqL
BcxrV/1n3xdzfY9FX6s4KGudo+gYsVpY0mnp2U8jG8YUNLDXQTXnRo4CQ0tRJJBa
RFDo2f5DqToZftuEhIsk+Mak1uXoA0eIYqGe61Xa/JEuwV1qLYubCNLUo+kgN03v
hkF8F7exFDAePw7guuvDvB0M0qBCaKDCsz1HoKRNVEduYrEQBJnX235Q4IsdI/
GcQ/dvERJXaDCG8EPhnuc517EMUJD1J1CXT4+VMHphmFbiVqmetz0upIJ+D037Xk
CcgXnte6L2orGRZ/11MYIN11GJKtCCFK7XGVPK1J8zyGSyPj1FfwtBy5hUX3aqtM
bvP0H2BRKoe1sbREnu58BKU6Yh1Ury7pVu1
=SJ1j
-----END PGP PUBLIC KEY BLOCK-----
socanalyst@ubuntu-victim:~$ sudo mkdir -p /usr/share/keyrings
socanalyst@ubuntu-victim:~$ sudo gpg --no-default-keyring --keyring gnupg-ring:/usr/share/keyrings/w
azuh.gpg --import /tmp/wazuh-key.gpg
gpg: keyring '/usr/share/keyrings/wazuh.gpg' created
gpg: directory '/root/.gnupg' created
gpg: /root/.gnupg/trustdb.gpg: trustdb created
gpg: key 96B3EE5F2911145: public key 'Wazuh.com (Wazuh Signing Key) <support@wazuh.com>' imported
gpg: Total number processed: 1
gpg: imported: 1
socanalyst@ubuntu-victim:~$ sudo chmod 644 /usr/share/keyrings/wazuh.gpg
socanalyst@ubuntu-victim:~$ ls -l /usr/share/keyrings/wazuh.gpg
-rw-r--r-- 1 root root 2279 Jun 26 08:45 /usr/share/keyrings/wazuh.gpg
socanalyst@ubuntu-victim:~$
```

Fig. 3: Successful GPG key import and apt update showing the Wazuh repository fetched without signature errors

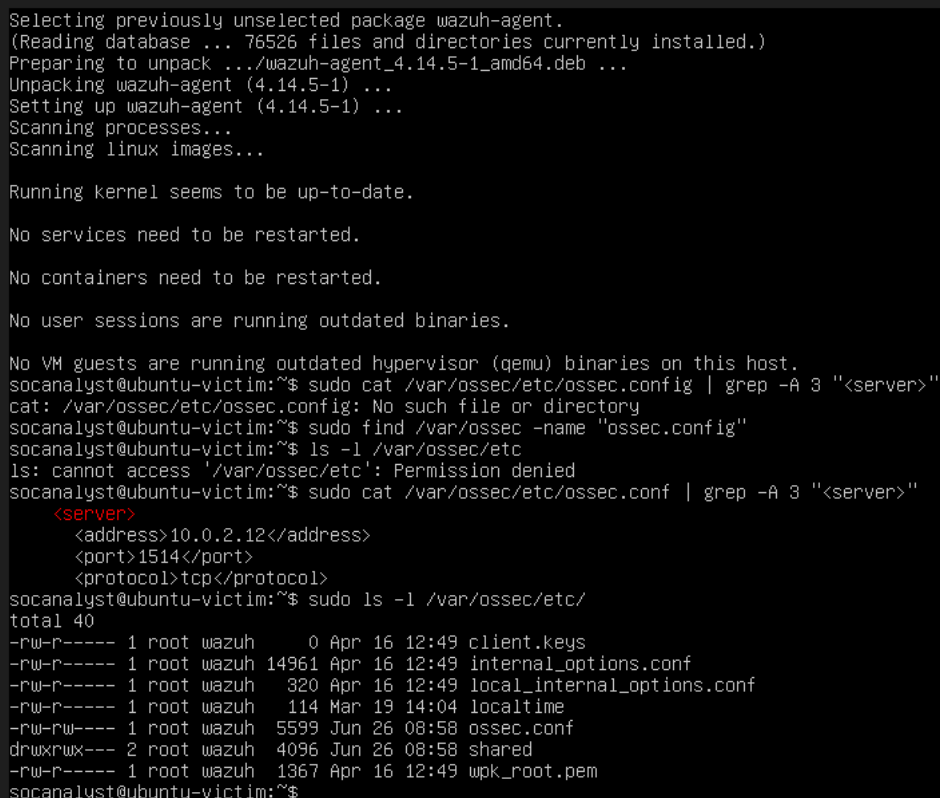
5.1.2 Agent Installation and Manager Linkage

The agent package was installed with the Wazuh Manager's IP address pre-configured via an environment variable, so that the manager address is written into `ossec.conf` automatically during installation:

```
sudo WAZUH_MANAGER="10.0.2.12" apt-get install wazuh-agent -y
```

This was verified by inspecting the resulting configuration:

```
sudo cat /var/ossec/etc/ossec.conf | grep -A 3 "<server>"
```



```
Selecting previously unselected package wazuh-agent.
(Reading database ... 76526 files and directories currently installed.)
Preparing to unpack .../wazuh-agent_4.14.5-1_amd64.deb ...
Unpacking wazuh-agent (4.14.5-1) ...
Setting up wazuh-agent (4.14.5-1) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
socanalyst@ubuntu-victim:~$ sudo cat /var/ossec/etc/ossec.config | grep -A 3 "<server>"
cat: /var/ossec/etc/ossec.config: No such file or directory
socanalyst@ubuntu-victim:~$ sudo find /var/ossec -name "ossec.config"
socanalyst@ubuntu-victim:~$ ls -l /var/ossec/etc
ls: cannot access '/var/ossec/etc': Permission denied
socanalyst@ubuntu-victim:~$ sudo cat /var/ossec/etc/ossec.conf | grep -A 3 "<server>"
<server>
  <address>10.0.2.12</address>
  <port>1514</port>
  <protocol>tcp</protocol>
socanalyst@ubuntu-victim:~$ sudo ls -l /var/ossec/etc/
total 40
-rw-r----- 1 root wazuh  0 Apr 16 12:49 client.keys
-rw-r----- 1 root wazuh 14961 Apr 16 12:49 internal_options.conf
-rw-r----- 1 root wazuh  320 Apr 16 12:49 local_internal_options.conf
-rw-r----- 1 root wazuh  114 Mar 19 14:04 localtime
-rw-rw---- 1 root wazuh  5599 Jun 26 08:58 ossec.conf
drwxrwx--- 2 root wazuh  4096 Jun 26 08:58 shared
-rw-r----- 1 root wazuh  1367 Apr 16 12:49 wpk_root.pem
socanalyst@ubuntu-victim:~$
```

Fig. 4: `ossec.conf` `<server>` block confirming the Manager IP (10.0.2.12) and port 1514/tcp

5.1.3 Agent Enrollment (Key Exchange)

Installing the agent software does not, by itself, authenticate the agent to the Manager. A manual key exchange was performed using Wazuh's `manage_agents` utility, run on both machines:

- On the Manager: `manage_agents` → (A)dd agent → specify name and IP → (E)xtract key for the new agent ID.
- On the Victim: `manage_agents` → (I)mport key → paste the extracted key string → confirm.

```
sudo systemctl restart wazuh-agent
```

Connectivity was confirmed from the Manager side:

```
sudo /var/ossec/bin/agent_control -l
```



```

(R)emove an agent (R).
(Q)uit.
Choose your action: A,E,L,R or Q: Q

manage_agents: Exiting.
[wazuh-user@wazuh-server ~]$ sudo /var/ossec/bin/agent_control -l
/var/ossec/bin/agent_control: invalid option -- 'l'

Wazuh agent_control: Control remote agents.
Available options:
  -h          This help message.
  -l          List available (active or not) agents.
  -lc         List active agents only.
  -ln         List disconnected agents only.
  -i <id>     Extracts information from an agent.
  -R -a       Restart all agents.
  -R -u <id>  Restart the specified agent.
  -r -a       Runs the integrity/rootkit checking on all agents now.
  -r -u <id>  Runs the integrity/rootkit checking on one agent now.

  -s          Changes the output to CSU (comma delimited).
  -j          Changes the output to JSON .

Available options for active response:
  -b <ip>     Blocks the specified ip address.
  -f <ar> -a   Used with -b, specifies which response to run. Apply AR on all agents.
  -f <ar> -u <id> Used with -b, specifies which response to run. Apply AR on specified agent
  .
  -L          List available active responses.
[wazuh-user@wazuh-server ~]$ sudo /var/ossec/bin/agent_control -l

Wazuh agent_control. List of available agents:
  ID: 000, Name: wazuh-server (server), IP: 127.0.0.1, Active/Local
  ID: 001, Name: ubuntu-victim, IP: any, Active

List of agentless devices:
[wazuh-user@wazuh-server ~]$

```

Fig. 5: `agent_control -l` output showing the `ubuntu-victim` agent with status 'Active'

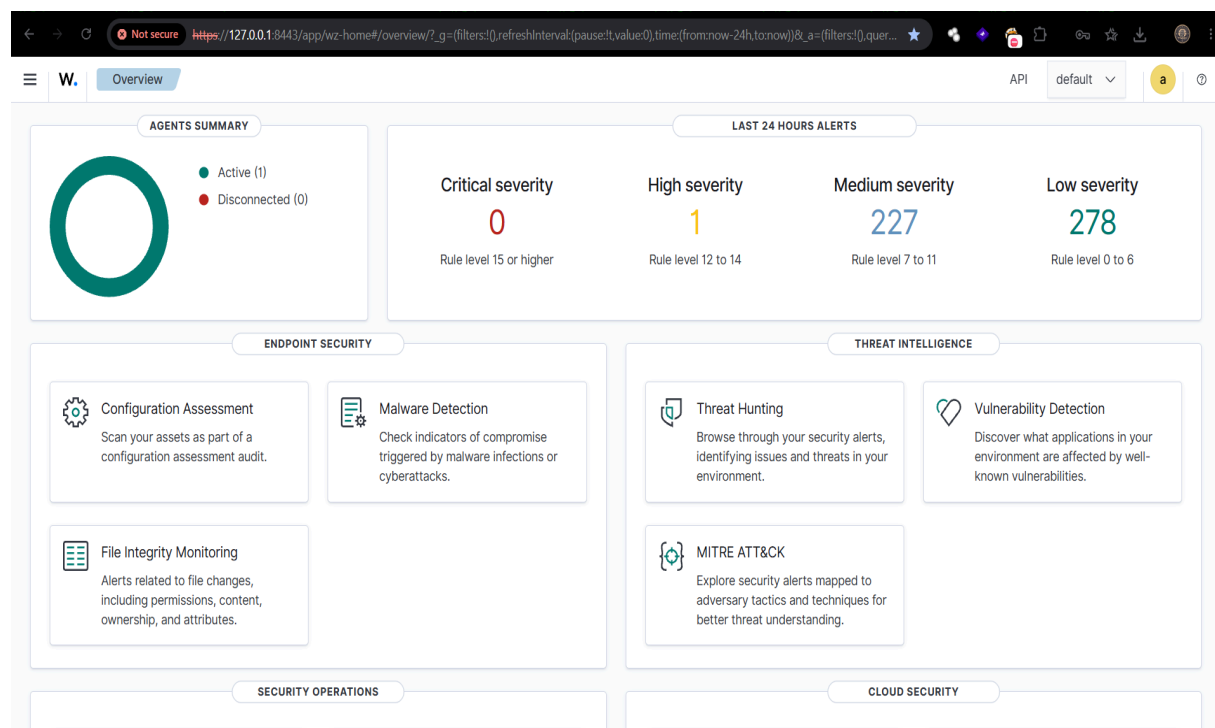


Fig. 6: Wazuh Dashboard Agents Summary widget showing Active (1), Disconnected (0)

5.2 Phase 2 — Wazuh ↔ Snort Log Integration

With the agent connected, the next phase was to make Wazuh aware of Snort's detections by configuring it to monitor Snort's alert log.

5.2.1 Identifying Snort's Output Format

Snort's configuration (/etc/snort/snort.conf) was inspected to determine its active logging outputs:

```
sudo grep -i "output" /etc/snort/snort.conf | grep -i alert
```

Three output methods were found configured simultaneously: a binary unified2 log, a syslog output, and a plain-text alert_fast log at /var/log/snort/snort.alert.fast. The alert_fast format was selected for integration, since it is human-readable, one alert per line, and matched by a decoder already built into Wazuh (snort-fast).

5.2.2 Configuring the Wazuh Agent to Monitor the Log

A <localfile> directive was added to the agent's ossec.conf:

```
<ossec_config>
  <localfile>
    <log_format>snort-fast</log_format>
    <location>/var/log/snort/snort.alert.fast</location>
  </localfile>
</ossec_config>
```

Note: Wazuh's ossec.conf permits multiple sequential <ossec_config> blocks within a single file. This directive was added as its own wrapped block rather than appended loose at the end of the file, to ensure valid XML structure (see Section 6.1 for the troubleshooting detail).

The agent was restarted to apply the change, and the agent's own log confirmed the new file was picked up:

```
sudo systemctl restart wazuh-agent
sudo tail -n 20 /var/ossec/logs/ossec.log
```

```

└─2081 /var/ossec/bin/wazuh-logcollector
└─2101 /var/ossec/bin/wazuh-modulesd

Jun 26 17:08:53 ubuntu-victim systemd[1]: Starting Wazuh agent...
Jun 26 17:08:53 ubuntu-victim env[2020]: Starting Wazuh v4.14.5...
Jun 26 17:08:54 ubuntu-victim env[2020]: Started wazuh-execd...
Jun 26 17:08:55 ubuntu-victim env[2020]: Started wazuh-agentd...
Jun 26 17:08:56 ubuntu-victim env[2020]: Started wazuh-syscheckd...
Jun 26 17:08:57 ubuntu-victim env[2020]: Started wazuh-logcollector...
Jun 26 17:08:58 ubuntu-victim env[2020]: Started wazuh-modulesd...
Jun 26 17:09:00 ubuntu-victim env[2020]: Completed.
Jun 26 17:09:00 ubuntu-victim systemd[1]: Started Wazuh agent.
socanalyst@ubuntu-victim:~$ sudo tail -n 20 /var/ossec/logs/ossec.log
2026/06/26 17:08:56 wazuh-logcollector: INFO: (1950): Analyzing file: '/var/log/snort/snort.alert.fast'.
2026/06/26 17:08:56 wazuh-logcollector: INFO: Started (pid: 2081).
2026/06/26 17:08:57 wazuh-syscheckd: INFO: (6009): File integrity monitoring scan ended.
2026/06/26 17:08:57 wazuh-syscheckd: INFO: FIM sync module started.
2026/06/26 17:08:57 wazuh-modulesd: INFO: Started (pid: 2101).
2026/06/26 17:08:57 wazuh-modulesd:ciscat: INFO: Module disabled. Exiting...
2026/06/26 17:08:57 wazuh-modulesd:osquery: INFO: Module disabled. Exiting...
2026/06/26 17:08:57 wazuh-modulesd:agent-upgrade: INFO: (8153): Module Agent Upgrade started.
2026/06/26 17:08:57 sca: INFO: Module started.
2026/06/26 17:08:57 wazuh-modulesd:control: INFO: Starting control thread.
2026/06/26 17:08:57 sca: INFO: Loaded policy '/var/ossec/ruleset/sca/cis_ubuntu22-04.yml'
2026/06/26 17:08:57 sca: INFO: Starting Security Configuration Assessment scan.
2026/06/26 17:08:57 wazuh-modulesd:syscollector: INFO: Module started.
2026/06/26 17:08:57 wazuh-modulesd:syscollector: INFO: Starting evaluation.
2026/06/26 17:08:57 sca: INFO: Starting evaluation of policy: '/var/ossec/ruleset/sca/cis_ubuntu22-04.yml'
2026/06/26 17:08:58 wazuh-logcollector: INFO: (9203): Monitoring journal entries.
2026/06/26 17:08:58 wazuh-modulesd:syscollector: INFO: Evaluation finished.
2026/06/26 17:09:03 sca: INFO: Evaluation finished for policy '/var/ossec/ruleset/sca/cis_ubuntu22-04.yml'
2026/06/26 17:09:03 sca: INFO: Security Configuration Assessment scan finished. Duration: 6 seconds.
2026/06/26 17:09:22 rootcheck: INFO: Ending rootcheck scan.
socanalyst@ubuntu-victim:~$

```

Fig. 7: *ossec.log* showing: *wazuh-logcollector: INFO: Analyzing file: '/var/log/snort/snort.alert.fast'*

5.3 Phase 3 — Enabling Active Detection and Generating Attack Traffic

5.3.1 Enabling the Portscan Preprocessor

An initial test scan produced no dedicated Snort alert. Inspection of `snort.conf` revealed that the `sfportscan` preprocessor — responsible for detecting reconnaissance-style scanning behavior — was commented out by default:

```
# preprocessor sfportscan: proto { all } memcap { 10000000 } sense_level { low }
```

This line was uncommented to activate scan detection:

```
preprocessor sfportscan: proto { all } memcap { 10000000 } sense_level { low }
sudo systemctl restart snort
```

5.3.2 Generating Attack Traffic

An Nmap TCP SYN scan was used as the test attack, as this scan technique is a classic reconnaissance pattern that intrusion detection systems are specifically designed to flag:

```
sudo nmap -sS 10.0.2.14
```

This command was ultimately run from the Wazuh-Manager VM against the Ubuntu-Victim VM, for reasons explained in Section 6.2.

6. Troubleshooting Log and Technical Findings

A SOC analyst's value lies as much in methodical troubleshooting as in initial setup. This section documents the most significant issues encountered during the build, the diagnostic process used to isolate each root cause, and the resolution applied. These are presented in the order encountered, as a genuine record of the engineering process rather than a sanitized happy-path account.

6.1 GPG Keyring Syntax Error

Symptom: Piping the downloaded GPG key directly into `gpg --import` produced repeated “general error” and “no writable keyring found” failures.

Root cause: A single-character typo (`gunpg-ring` instead of `gnupg-ring`) in the `--keyring` argument caused `gpg` to reject the resource string before it ever attempted to write the keyring file.

Resolution: The key download was separated from the import step (`curl -o file`, then `gpg --import file`) to isolate network issues from syntax issues, making the typo immediately visible in the error output.

Lesson: Splitting a piped command into discrete steps is a reliable debugging technique when a compound command fails opaquely — it narrows the failure to a single, inspectable stage.

6.2 Self-Directed Scan Traffic Invisible to Snort

Symptom: An Nmap SYN scan run on the Ubuntu-Victim machine, targeting its own IP address (10.0.2.14), produced zero Snort alerts — despite Snort being confirmed active and the portscan preprocessor being enabled.

Diagnostic process: Rather than continuing to adjust Snort's configuration, the routing path of the scan traffic itself was inspected:

```
ip route get 10.0.2.14
# Result: local 10.0.2.14 dev lo src 10.0.2.14 uid 1000
```

This confirmed that traffic addressed to the machine's own IP is routed by the Linux kernel through the loopback interface (`lo`), not the physical network interface. Snort, however, was confirmed (via its startup command, captured from `systemctl status snort`) to be listening specifically on the real interface:

```
/usr/sbin/snort -i enp0s3 -c /etc/snort/snort.conf ...
```

Root cause: Self-targeted traffic never crosses the `enp0s3` interface that Snort monitors; it is resolved entirely within the kernel via loopback. This is not a Snort or Wazuh configuration issue — it is fundamental to how the Linux network stack handles locally-addressed traffic, and no IDS rule tuning can change it.

Resolution: Attack traffic was instead generated from a separate host — the Wazuh-Manager VM — targeting the victim. This produced genuine cross-host traffic that correctly traversed the monitored interface and was detected.

Operational context: This finding also explains why the original plan to use a dedicated Kali Linux attacker VM was the architecturally correct design — host resource constraints (see below) made it

temporarily impractical, but the underlying networking principle confirms that IDS testing must always originate from a genuinely separate network endpoint.

6.3 Host RAM Constraints and the Kali Linux Attacker VM

Symptom: Attempting to run all three virtual machines (Wazuh-Manager, Ubuntu-Victim, and Kali Linux) simultaneously caused the host laptop to freeze.

Diagnosis: Windows Task Manager confirmed the host has only 8 GB of total physical RAM. With Wazuh-Manager (3072 MB) and Ubuntu-Victim (1536 MB) already running, available memory was measured at only 261 MB — far below what a third VM (even after reducing Kali’s allocation to 1536 MB) would require to boot reliably. The system was found to be relying heavily on disk-based virtual memory (page file), which is dramatically slower than physical RAM and was the direct cause of the freeze.

Resolution: Per-VM resource allocations were reduced where safe to do so (see Section 3.2), and the Kali Linux VM was ultimately not run concurrently with the other two machines. Attack traffic was generated instead from the Wazuh-Manager VM (see Section 6.2), which required no additional RAM since that machine was already running.

Lesson: In resource-constrained lab environments, the specific machine that originates test traffic is less important than ensuring the traffic genuinely crosses a real network boundary. Documenting this constraint and the reasoning behind the workaround demonstrates engineering judgment rather than weakening the project’s technical validity.

6.4 Malformed ossec.conf XML Structure

Symptom: After adding the `<localfile>` block for Snort monitoring, the new directive was found sitting immediately after a closing `</ossec_config>` tag, rather than inside any configuration block.

Root cause: Wazuh’s `ossec.conf` supports multiple concatenated `<ossec_config>...</ossec_config>` blocks in a single file. Navigating to “end of file” in a text editor placed the cursor after the final closing tag of the last existing block, rather than inside it.

Resolution: The new directive was wrapped in its own dedicated `<ossec_config> ... </ossec_config>` pair, consistent with the pattern used elsewhere in the file, and verified with a targeted `tail -n 10` check before restarting the agent.

6.5 Wazuh-Indexer (OpenSearch) Out-of-Memory Crash

Symptom: Dashboard login began failing with “Invalid username or password,” despite previously-working credentials. Attempting to reset the admin password via `wazuh-passwords-tool.sh` failed with “ERR: Seems there is no OpenSearch running on 127.0.0.1:9200.”

Diagnosis: `systemctl status wazuh-indexer` revealed the service had failed with Result: `oom-kill` — the Linux kernel had forcibly terminated the OpenSearch process due to memory exhaustion on the Wazuh-Manager VM.

Resolution: The indexer service was restarted (`systemctl start wazuh-indexer`) and confirmed healthy before re-attempting the password reset. Free memory was checked (`free -h`) immediately afterward and found critically low (229 MB available), confirming the service remained at risk and should not be left under sustained load.

Secondary issue: The chosen replacement password (containing the symbol `@`) was rejected by Wazuh’s password complexity policy, which only accepts symbols from the set `. * + ? -`. A compliant password (Soclab.2026) was used instead.

Lesson: A generic-looking authentication failure can have a completely unrelated infrastructure-level root cause. Checking the health of dependent services (the database/indexer backing the login system) before assuming the credentials themselves are wrong saves significant troubleshooting time once applied.

6.6 Agent Re-enrollment Producing a Duplicate Identity

Symptom: After a VM reboot, the Wazuh Dashboard showed the agent as “Disconnected,” and the agent’s log showed repeated “ERROR: Duplicate agent name: ubuntu-victim. Unable to add agent (from manager)” messages.

Diagnosis: Inspecting `/var/ossec/etc/client.keys` on the victim revealed the agent was now registered under ID 002, while the manager’s `manage_agents → (L)ist` command was checked to confirm the manager’s authoritative current record.

Resolution: The manager’s agent list was treated as the single source of truth. Once confirmed that only ID 002 existed there (matching the victim’s key file), the agent was simply restarted; `agent_control -l` then confirmed status Active.

7. Results and Verification

With all infrastructure issues resolved, the complete detection pipeline was validated using a live attack test. This section presents the alert as it appeared at each stage of the pipeline.

7.1 The Test Attack

A TCP SYN scan was launched from the Wazuh-Manager VM (10.0.2.12) against the Ubuntu-Victim VM (10.0.2.14):

```
sudo nmap -sS 10.0.2.14
```

```
Run the following command to upgrade to 2023.12.20260622:

dnf upgrade --releasever=2023.12.20260622

Release notes:
https://docs.aws.amazon.com/linux/al2023/release-notes/relnotes-2023.12.20260622.html

=====

Installed:
  libssh2-1.10.0-1.amzn2023.0.3.x86_64                nmap-3:7.93-4.amzn2023.x86_64
  nmap-ncat-3:7.93-4.amzn2023.x86_64

Complete!
[wazuh-user@wazuh-server ~]$ ping -c 10.0.2.14
ping: invalid argument: '10.0.2.14'
[wazuh-user@wazuh-server ~]$ ping -c 4 10.0.2.14
PING 10.0.2.14 (10.0.2.14) 56(84) bytes of data:
64 bytes from 10.0.2.14: icmp_seq=1 ttl=64 time=0.387 ms
64 bytes from 10.0.2.14: icmp_seq=2 ttl=64 time=0.277 ms
64 bytes from 10.0.2.14: icmp_seq=3 ttl=64 time=0.271 ms
64 bytes from 10.0.2.14: icmp_seq=4 ttl=64 time=0.257 ms

--- 10.0.2.14 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3064ms
rtt min/avg/max/mdev = 0.257/0.298/0.387/0.051 ms
[wazuh-user@wazuh-server ~]$ sudo nmap -sS 10.0.2.14
Starting Nmap 7.93 ( https://nmap.org ) at 2026-06-27 03:39 UTC
Nmap scan report for 10.0.2.14
Host is up (0.00014s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
MAC Address: 08:00:27:8C:52:3E (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.40 seconds
[wazuh-user@wazuh-server ~]$
```

Fig. 8: Nmap scan executed from Wazuh-Manager, showing successful connectivity and scan completion

7.2 Stage 1 — Raw Snort Detection

Immediately after the scan, Snort's alert log showed two new entries, with timestamps matching the scan and the correct source/destination IP pair:

```
06/27-03:39:18.627538 [**] [1:1421:11] SNMP AgentX/tcp request [**]
[Classification: Attempted Information Leak] [Priority: 2]
{TCP} 10.0.2.12:63839 -> 10.0.2.14:705

06/27-03:39:18.632924 [**] [1:1418:11] SNMP request tcp [**]
[Classification: Attempted Information Leak] [Priority: 2]
{TCP} 10.0.2.12:63839 -> 10.0.2.14:161
```

```

Traffic] [Priority: 2] {UDP} 0.0.0.0:68 -> 255.255.255.255:67
06/26-02:20:22.344154  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad
Traffic] [Priority: 2] {UDP} 0.0.0.0:68 -> 255.255.255.255:67
06/26-02:20:22.727904  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad
Traffic] [Priority: 2] {IPV6-ICMP} :: -> ff02::16
06/26-02:20:23.137475  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad
Traffic] [Priority: 2] {IPV6-ICMP} :: -> ff02::1:ffb4:c5d2
socanalyst@ubuntu-victim:~$ sudo grep "06/27" /var/log/snort/snort.alert.fast | tail -n 20
socanalyst@ubuntu-victim:~$ sudo wc -l /var/log/snort/snort.alert.fast
5 /var/log/snort/snort.alert.fast
socanalyst@ubuntu-victim:~$ date
Sat Jun 27 03:29:08 AM UTC 2026
socanalyst@ubuntu-victim:~$ sudo systemctl status snort | grep -i "snort -m"
└─2906 /usr/sbin/snort -m 027 -D -d -l /var/log/snort -u snort -g snort --pid-path /run
/var/log/snort -c /etc/snort/snort.conf -S "\$HOME_NET=[10.0.2.0/24]" -i enp0s3
socanalyst@ubuntu-victim:~$ sudo grep -i "config interface\^-i" /etc/snort/snort.conf
socanalyst@ubuntu-victim:~$ ip route get 10.0.2.14
local 10.0.2.14 dev lo src 10.0.2.14 uid 1000
    cache <local>
socanalyst@ubuntu-victim:~$ sudo tail -n 20 /var/log/snort/snot.alert.fast
tail: cannot open '/var/log/snort/snot.alert.fast' for reading: No such file or directory
socanalyst@ubuntu-victim:~$ sudo tail -n 20 /var/log/snort/snort.alert.fast
06/26-02:20:23.328591  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad
Traffic] [Priority: 2] {IPV6-ICMP} :: -> ff02::16
06/26-02:20:22.340409  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad
Traffic] [Priority: 2] {UDP} 0.0.0.0:68 -> 255.255.255.255:67
06/26-02:20:22.344154  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad
Traffic] [Priority: 2] {UDP} 0.0.0.0:68 -> 255.255.255.255:67
06/26-02:20:22.727904  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad
Traffic] [Priority: 2] {IPV6-ICMP} :: -> ff02::16
06/26-02:20:23.137475  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad
Traffic] [Priority: 2] {IPV6-ICMP} :: -> ff02::1:ffb4:c5d2
06/27-03:39:18.627538  [**] [1:1421:11] SNMP AgentX/tcp request [**] [Classification: Attempted Info
rmation Leak] [Priority: 2] {TCP} 10.0.2.12:63839 -> 10.0.2.14:705
06/27-03:39:18.632924  [**] [1:1418:11] SNMP request tcp [**] [Classification: Attempted Information
Leak] [Priority: 2] {TCP} 10.0.2.12:63839 -> 10.0.2.14:161
socanalyst@ubuntu-victim:~$ _

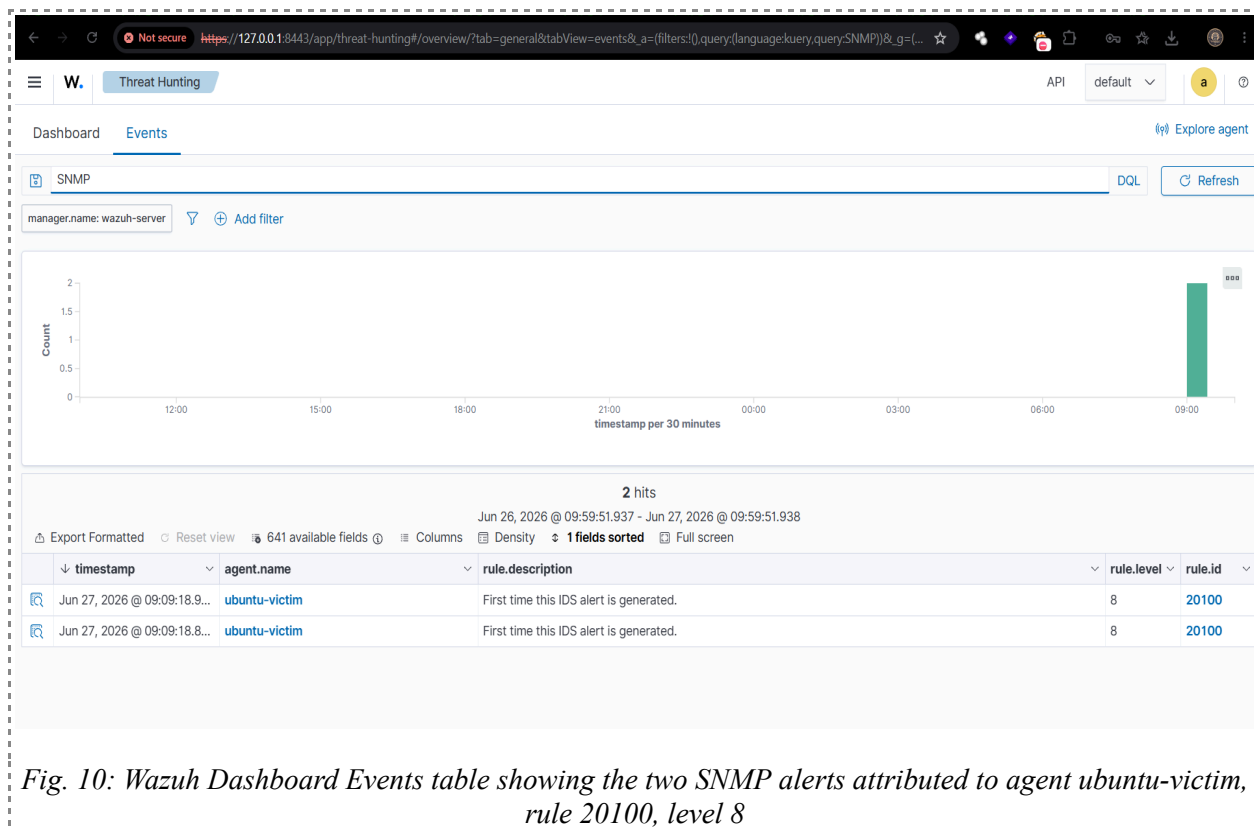
```

Fig. 9: *snort.alert.fast* showing the two SNMP-classified detections with correct source (10.0.2.12) and destination (10.0.2.14) IPs

7.3 Stage 2 — Wazuh Dashboard Visualization

The same event was located in the Wazuh Dashboard under Threat Hunting → Events, filtered by the keyword SNMP:

Field	Value
Timestamp	Jun 27, 2026 @ 09:09:18
agent.name	ubuntu-victim
rule.description	First time this IDS alert is generated.
rule.level	8
rule.id	20100
Hits returned	2



Rule 20100 is one of Wazuh’s built-in meta-rules, specifically designed to flag the first occurrence of any alert originating from an integrated IDS data source. Its presence in the results, combined with the correct agent attribution, confirms that Wazuh correctly identified, parsed, and classified the Snort-sourced event — rather than treating it as an unrecognized generic log line.

8. SOC-Style Alert Analysis

Presented in the standard SOC alert-triage format, for documentation and viva reference:

Field	Detail
Alert Name	SNMP AgentX/tcp request / SNMP request tcp
Severity	Medium (Wazuh rule level 8; Snort Priority 2)
Source IP	10.0.2.12 (Wazuh-Manager, used as test traffic origin)
Destination IP	10.0.2.14 (Ubuntu-Victim, monitored endpoint)
MITRE ATT&CK Mapping	T1046 – Network Service Discovery; T1040 – Network Sniffing (associated with SNMP reconnaissance)
Impact	Indicates reconnaissance activity probing for an exposed SNMP service, which — if successfully queried — can leak device configuration, routing, and system information useful for further attack planning.
Recommended Action	Verify whether SNMP is required on the host; if not, disable the service. If required, restrict access via firewall rules to known management hosts, enforce SNMPv3 with authentication/encryption, and continue monitoring for repeated probing from the same source.

9. Security Best Practices Demonstrated

- Least-privilege service accounts: Wazuh’s agent and indexer processes run under dedicated, non-root system users (wazuh).
- Cryptographic package verification: all software installed via APT was verified against an imported GPG signing key before installation, preventing supply-chain tampering.
- Authenticated agent enrollment: the Wazuh Agent cannot transmit data to the Manager without first completing a pre-shared key exchange, preventing rogue or spoofed agents from injecting false telemetry.
- Restricted configuration file permissions: sensitive files such as client.keys are owned by root/wazuh and inaccessible to standard users.
- Password complexity enforcement: the Wazuh platform itself enforces a minimum password policy (length, case, numeral, symbol) on its administrative accounts.
- Centralized, tamper-resistant logging: once forwarded to the Manager, alert data is stored independently of the originating host, so a compromised endpoint cannot retroactively erase evidence of its own compromise.

10. Real-World SOC Relevance

Every component of this lab mirrors a function performed in production SOC environments at far larger scale:

- Agent-based telemetry collection (Wazuh Agent) is the same architectural pattern used by commercial EDR/XDR platforms to gather host and network signals from thousands of endpoints.
- Network-layer IDS (Snort) represents the detection layer that commonly sits inline or as a tap on critical network segments in enterprise environments, often complemented by commercial NGFW/IPS appliances running similar signature-based logic.
- Centralized correlation and rule-based alerting (Wazuh Manager) mirrors the core function of any SIEM — Splunk, Microsoft Sentinel, Elastic Security — turning raw log volume into prioritized, actionable alerts.
- MITRE ATT&CK mapping, as demonstrated in Section 8, is standard SOC practice for contextualizing an individual alert within a broader adversary tactic, supporting faster triage and threat-hunting decisions.
- The troubleshooting process itself — diagnosing service failures, resource exhaustion, and configuration errors under time pressure — reflects the day-to-day reality of SOC and SIEM engineering work, where infrastructure reliability is as important as detection logic.

11. Conclusion

This project successfully achieved its core objective: building and validating a working integration between Snort and Wazuh for centralized intrusion detection and monitoring. Despite a series of genuine infrastructure and configuration obstacles — GPG signing errors, host memory exhaustion, a non-obvious Linux loopback routing behavior, malformed XML configuration, and an OpenSearch crash — each issue was systematically diagnosed and resolved using log inspection, first-principles reasoning, and targeted verification at every stage.

The final result is a fully evidenced, end-to-end detection pipeline: a real network attack, originating from a genuinely separate host, was detected by Snort at the packet level, correctly classified, forwarded through an authenticated Wazuh Agent, correlated by the Wazuh Manager, and rendered as a structured, attributable event in the Wazuh Dashboard. This outcome, together with the documented troubleshooting record, demonstrates both the technical integration itself and the operational problem-solving skills central to real-world SOC and SIEM engineering work.

11.1 Future Improvements

- **Expand testing**, using a dedicated Kali Linux attacker VM and custom Snort rules to cover a broader range of attacks beyond a single scan type.
- **Harden infrastructure**, increasing Wazuh-Manager's memory allocation to prevent OpenSearch crashes under load.
- **Extend to automated response**, using Wazuh's Active Response module to auto-block detected attacker IPs.

12. Appendix: Figure Index

The following screenshots are referenced throughout this report and should be inserted at their corresponding placeholder locations.

Figure	Description
Fig. 1	Wazuh Manager network interface configuration (10.0.2.12)
Fig. 2	Ubuntu Victim network interface configuration (10.0.2.14)
Fig. 3	Successful Wazuh GPG key import and apt update
Fig. 4	ossec.conf <server> block confirming Manager IP and port
Fig. 5	agent_control -l output — agent status Active
Fig. 6	Wazuh Dashboard Agents Summary — Active (1)
Fig. 7	ossec.log confirming Snort log file is being monitored
Fig. 8	Nmap SYN scan executed from Wazuh-Manager
Fig. 9	snort.alert.fast — raw SNMP detection entries
Fig. 10	Wazuh Dashboard Events table — SNMP alert, rule 20100